

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Оглавление

Исключения (exceptions)	2
Что такое исключения?	2
Иерархия классов для встроенных исключений	4
Перехват исключений	5
Блок try	6
Блок except	6
Блок else	7
Блок finally	8

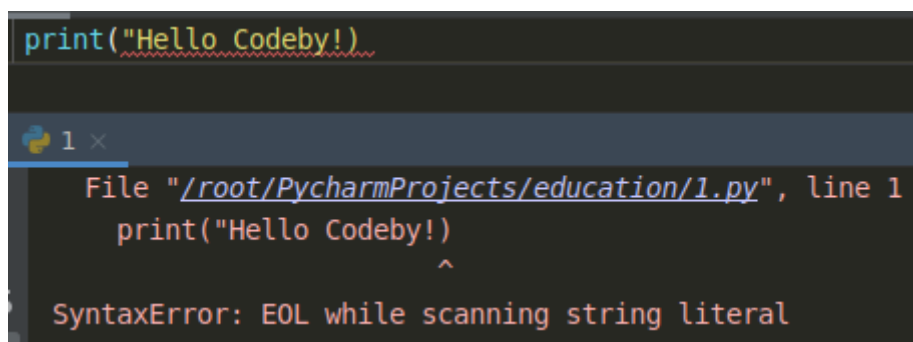
Исключения (exceptions)

Что такое исключения?

Исключительные ситуации или исключения (exceptions) – это ошибки, обнаруженные при выполнении кода программы. Например, попытка чтения несуществующего файла или деления на ноль, приведёт к тому, что Python не сможет обработать эти ситуации, и выдаст [Traceback](#) или на сленге – трэйс – путь к месту и информацию о месте появления ошибки. По трэйсу найти и исправить ошибку становится проще.

Рассмотрим разные типы ошибок:

Синтаксические – возникают из-за синтаксических погрешностей кода

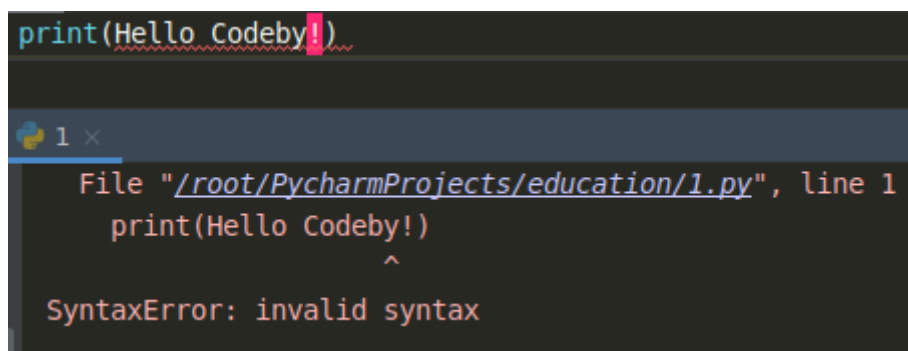


```
print('Hello Codeby!')  
  
1 x  
File "/root/PycharmProjects/education/1.py", line 1  
    print('Hello Codeby!')  
        ^  
SyntaxError: EOL while scanning string literal
```

Трейс сообщает, что в первой строке кода есть ошибка, и выводит краткую информацию, что при сканировании строкового литерала обнаружена синтаксическая ошибка. Более того, указывается знаком ^ на место возникновения ошибки.

Согласитесь, это круто облегчает поиск и устранение багов. В данном примере, строка не закрыта кавычками. Точно такую же ошибку мы бы получили, если у строки были бы разные кавычки.

Разновидность синтаксической ошибки - [invalid syntax](#) получится, если вовсе забыть оформить строку кавычками.

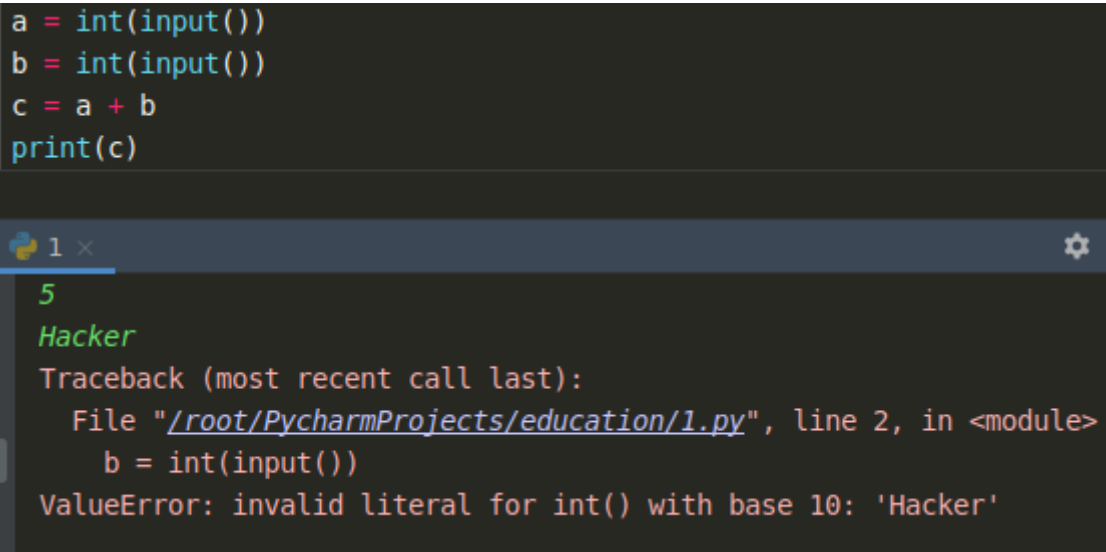


```
print(Hello Codeby!)  
  
1 x  
File "/root/PycharmProjects/education/1.py", line 1  
    print(Hello Codeby!)  
        ^  
SyntaxError: invalid syntax
```

Exceptions (исключения) - представляют собой еще один вид ошибок, который проявляется в ходе выполнения программы.

Исключения могут вызываться из-за ошибочных действий пользователя, не предусмотренных программой.

```
a = int(input())
b = int(input())
c = a + b
print(c)
```

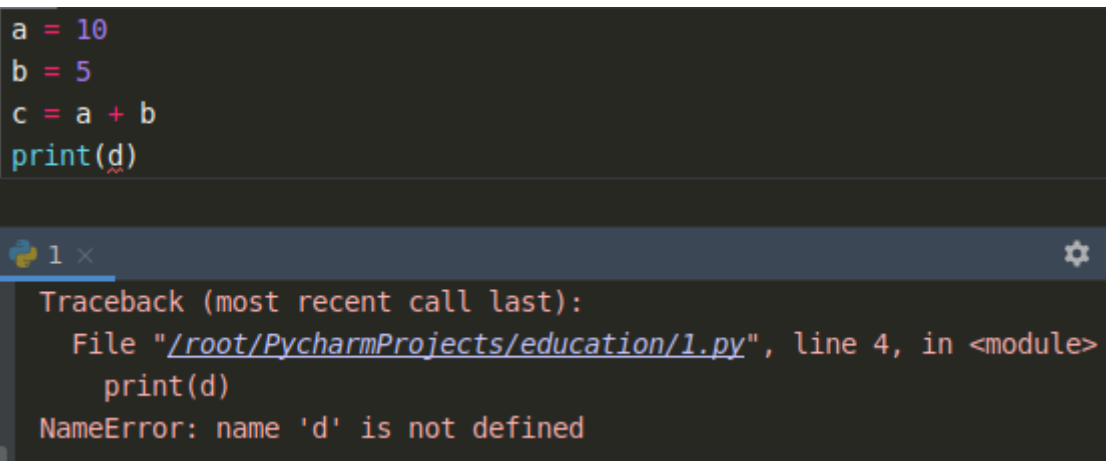


```
1 x
5
Hacker
Traceback (most recent call last):
  File "/root/PycharmProjects/education/1.py", line 2, in <module>
    b = int(input())
ValueError: invalid literal for int() with base 10: 'Hacker'
```

Данная программа предусматривала сложение целых чисел, но юзер ввёл только одно число, а вместо второго вписал строку, и программа сломалась, выдав исключение **ValueError** – ошибка типа. Исключение **ValueError** возникает, когда операция или функция получает объект с недопустимым значением.

Также исключения возникают при написании некорректного кода программы, верного синтаксически.

```
a = 10
b = 5
c = a + b
print(d)
```



```
1 x
Traceback (most recent call last):
  File "/root/PycharmProjects/education/1.py", line 4, in <module>
    print(d)
NameError: name 'd' is not defined
```

В данном примере возникает исключение **NameError** сообщающее, что имя 'd' не объявлено. Программист случайно нажал на соседнюю клавишу, и вместо 'c' напечатал 'd'. Исключение **NameError** возникает, когда локальное или глобальное имя не найдено.

Разновидностей исключений огромное количество, и они имеют свою иерархию.

Иерархия классов для встроенных исключений:

```
BaseException
+-- SystemExit
+-- KeyboardInterrupt
+-- GeneratorExit
+-- Exception
    +-- StopIteration
    +-- StopAsyncIteration
    +-- ArithmeticError
    |   +-- FloatingPointError
    |   +-- OverflowError
    |   +-- ZeroDivisionError
    +-- AssertionError
    +-- AttributeError
    +-- BufferError
    +-- EOFError
    +-- ImportError
    |   +-- ModuleNotFoundError
    +-- LookupError
    |   +-- IndexError
    |   +-- KeyError
    +-- MemoryError
    +-- NameError
    |   +-- UnboundLocalError
    +-- OSError
    |   +-- BlockingIOError
    |   +-- ChildProcessError
    |   +-- ConnectionError
    |       +-- BrokenPipeError
    |       +-- ConnectionAbortedError
    |       +-- ConnectionRefusedError
    |       +-- ConnectionResetError
    |   +-- FileExistsError
    |   +-- FileNotFoundError
    |   +-- InterruptedError
    |   +-- IsADirectoryError
    |   +-- NotADirectoryError
    |   +-- PermissionError
    |   +-- ProcessLookupError
    |   +-- TimeoutError
    +-- ReferenceError
    +-- RuntimeError
    |   +-- NotImplementedError
    |   +-- RecursionError
    +-- SyntaxError
    |   +-- IndentationError
    |       +-- TabError
    +-- SystemError
    +-- TypeError
```

```

+-- ValueError
| +-- UnicodeError
|   +-- UnicodeDecodeError
|   +-- UnicodeEncodeError
|   +-- UnicodeTranslateError
+-- Warning
    +-- DeprecationWarning
    +-- PendingDeprecationWarning
    +-- RuntimeWarning
    +-- SyntaxWarning
    +-- UserWarning
    +-- FutureWarning
    +-- ImportWarning
    +-- UnicodeWarning
    +-- BytesWarning
    +-- ResourceWarning

```

Пояснение некоторых исключений:

BaseException - базовое исключение, от которого берут начало все остальные

FileExistsError - попытка создания файла, который уже существует

FileNotFoundError - файл или директория не существует

InterruptedError - системный вызов прерван входящим сигналом

IsADirectoryError - ожидался файл, но это директория

NotADirectoryError - ожидалась директория, но это файл

PermissionError - не хватает прав доступа

ProcessLookupError - указанного процесса не существует

TimeoutError - закончилось время ожидания

Полное описание исключений, можно посмотреть по [ссылке](#) на странице документации

Перехват исключений

Обработка исключений нужна для того, чтобы приложение не завершалось аварийно каждый раз, когда возникает исключение. Для этого блок кода, в котором возможно появление исключительной ситуации, необходимо поместить во внутрь конструкции `try...except`.

Работа операторов `try` `except` очень похожа на `if else`. Они точно также образуют блоки после двоеточия с 4-мя пробелами, в которых уже производятся какие-то действия. При этом все обычные команды помещаются внутрь `try`-блока, а все обработчики исключений – в `except`-блок.

Блок try

Если в ходе исполнения данного блока исключений не будет, то ни один из обработчиков не задействуется.

Если в ходе исполнения инструкций в этом блоке будет зафиксировано исключение, то начнётся поиск подходящего для него обработчика. Поиск производится по блокам `except` поочерёдно, поэтому более широкие типы исключений (базовые) имеет смысл ставить после более конкретных.

Блок `except`

Таких блоков может быть несколько и в каждом можно определить свой механизм обработки для того или иного типа исключения (или группы исключений). Все блоки `except` должны иметь тело, в котором реализуется обработка исключения.

Если блок `except` используется без следующего за ним выражения, то он должен быть последним, потому что в нём производится обработка любых типов. Но этот блок не должен быть единственным!

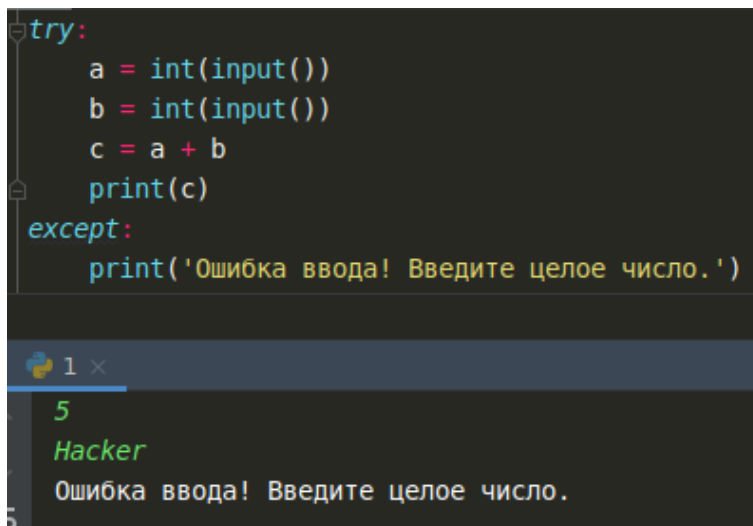
Вы можете использовать исключение более самого уровня `Exception`, чтобы перехватить все типы исключения.

```
except Exception as e:
    print(e)
```

Если за блоком следует выражение, то оно будет выполнено. Если полученный в результате выполнения объект совместим с исключением, которые мы собираемся обработать, то будут выполнены инструкции, находящиеся в данном блоке. Объект считается совместимым с поднятым исключением, если является классом (непосредственным, либо базовым) исключения, либо кортежем, содержащим элемент, совместимый с исключением.

Если в ходе поиска обработчик найден не будет, то поиск будет продолжен среди обработчиков окружающего кода.

Чтобы программа работала корректно, необходимо предусмотреть все неправильные действия пользователя, и иные ситуации.



```
try:
    a = int(input())
    b = int(input())
    c = a + b
    print(c)
except:
    print('Ошибка ввода! Введите целое число.')
```

1 x

5

Hacker

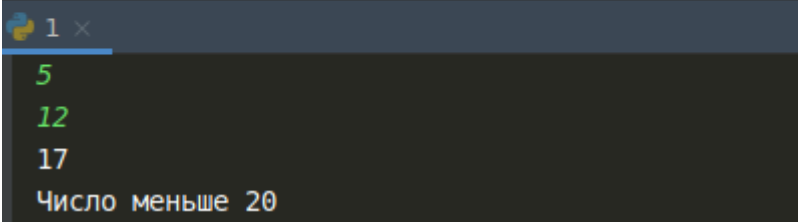
Ошибка ввода! Введите целое число.

Как видно, попытка ввода некорректного значения не удалась, так как возникшее исключение было перехвачено, и обработчик вывел пользовательское предупреждение.

Блок else

Инструкции в этом необязательном блоке выполняются по завершению блока try, если не возникло исключений. При этом в нём могут находиться вложенные операторы if, elif, else.

```
try:
    a = int(input())
    b = int(input())
    c = a + b
    print(c)
except:
    print('Ошибка ввода! Введите целое число.')
else:
    if c > 20:
        print('Число больше 20')
    else:
        print('Число меньше 20')
```



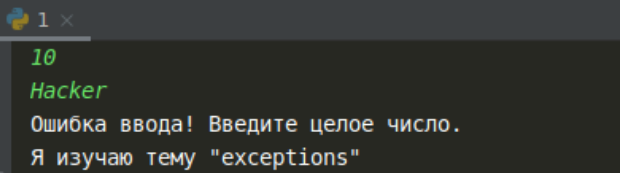
1 x

5
12
17
Число меньше 20

Блок finally

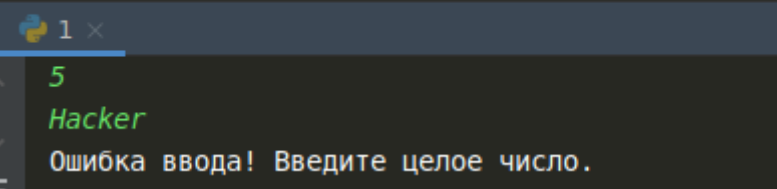
Этот блок не является обязательным, и выполнится в любом случае, вне зависимости было перехвачено исключение, или нет.

```
try:
    a = int(input())
    b = int(input())
    c = a + b
    print(c)
except:
    print('Ошибка ввода! Введите целое число.')
else:
    if c > 20:
        print('Число больше 20')
    else:
        print('Число меньше 20')
finally:
    print('Я изучаю тему "exceptions"')
```



Во всех приведённых выше примерах мы использовали базовое исключение. То есть не указывали какое конкретно исключение мы хотим обработать. Наиболее корректным вариантом всегда будет перехват определённого исключения. В данном случае это ValueError.

```
try:
    a = int(input())
    b = int(input())
    c = a + b
    print(c)
except ValueError:
    print('Ошибка ввода! Введите целое число.')
```



Результат работы аналогичен, но обработчик сразу же проверял именно это исключение, а не искал подходящее.

Нажатие Ctrl + D говорит терминалу, что надо зарегистрировать так называемый EOF (end of file – конец файла), то есть поток ввода окончен. Bash интерпретирует это как желание выйти из программы. Поэтому если пользователь нажмёт Ctrl + D, во время выполнения программы, то мы получим исключение EOFError.

```
try:
    a = int(input())
    b = int(input())
    c = a + b
    print(c)
except ValueError:
    print('Ошибка ввода! Введите целое число.')
```

```
1 x
Traceback (most recent call last):
  File "/root/PycharmProjects/education/1.py", line 2, in <module>
    a = int(input())
EOFError: EOF when reading a line
```

Для указания набора исключений, который должен обрабатывать данный блок except, их необходимо перечислить в круглых скобках через запятую после оператора except.

`except(ValueError, EOFError):`

Если мы хотим вывести разные сообщения результата обработки исключений (что бывает очень разумно), то нужно использовать несколько блоков except.

```
try:
    a = int(input())
    b = int(input())
    c = a + b
except ValueError:
    print('Ошибка ввода! Введите целое число.')
except EOFError:
    print('Отмена операции')
else:
    print(c)
```

Однако нужно учитывать, что в случае, если обработчик исключения так и не будет найден, то исключение будет необработанным (unhandled exception) и программа аварийно остановится.

Таким образом, если вы не уверены в том, какое/какие исключения нужно обработать, то используйте базовый вариант для подстраховки.

```
try:
    a = int(input())
    b = int(input())
    c = a + b
except ValueError:
    print('Ошибка ввода! Введите целое число.')
except:
    print('Ошибка ввода! Введите целое число.')
else:
    print(c)
```